Chapter 2

Using Functional Programming Techniques

IN THIS CHAPTER

- **Understanding functional programming concepts**
- **Using Python to perform functional programming tasks**
- **Performing essential functional programming tasks**
- **Relying on functional programming for data manipulation**

This chapter isn't about a specific programming language (even though it uses Python to present examples); it's about a programming paradigm. A *paradigm* is a framework that expresses a particular set of assumptions, relies on particular ways of thinking through problems, and uses particular methodologies to solve those problems. Other paradigms you may use are imperative, procedural, object-oriented, and declarative. Consequently, this chapter is different because it focuses on the problems you need to solve. The first part of this chapter discusses how the functional programming paradigm accomplishes this problem solving, and the second part points out how functional programming differs from other paradigms you may have used.

Throughout this chapter, you consider why you'd want to use functional programming at all. The math orientation of functional pro-

gramming means that you might not create an application using it; you might instead solve straightforward math problems or devise *what if* scenarios to test. Because functional programming is unique in its approach to solving problems, you might wonder how it actually accomplishes its goals. The third part looks at essential functional programming methods. Finally, the fourth part considers how you use functional programming for data manipulation, which is, of course, the topic of this book.

REMEMBER You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the `DSPD_0202_Functional.ipynb` source code file for Python source code file for Python and the `DSPD_R_0202_Functional.ipynb` source code file for R. See the Introduction for details on how to find these source files.

Defining Functional Programming

Functional programming has somewhat different goals and approaches than other paradigms use. Goals define what the functional programming paradigm is trying to do in forging the approaches used by languages that support it. However, the goals don't specify a particular implementation; doing that is within the purview of the individual languages. The following sections give you additional information on how the functional programming paradigm differs.

Differences with other programming paradigms

The main difference between the functional programming paradigm and other paradigms is that functional programs use math functions rather than statements to express ideas. This difference means that rather than write a precise set of steps to solve a problem, you use math functions, and you don't worry about how the language per-

forms the task. In some respects, this approach makes languages that support the functional programming paradigm similar to applications such as MATLAB. Of course, with MATLAB, you get a user interface, which reduces the learning curve. However, you pay for the convenience of the user interface with a loss of power and flexibility, which functional languages do offer. Using this approach to defining a problem relies on the *declarative programming* style, which you see used with other paradigms and languages, such as Structured Query Language (SQL) for database management.

In contrast to other paradigms, the functional programming paradigm doesn't maintain state. The use of *state* enables you to track values between function calls. Other paradigms use state to produce variant results based on environment, such as determining the number of existing objects and doing something different when the number of objects is zero. As a result, calling a functional program function always produces the same result given a particular set of inputs, thereby making functional programs more predictable than those that support state.

Because functional programs don't maintain state, the data they work with is also *immutable,* which means that you can't change it. To change a variable's value, you must create a new variable. Again, this makes functional programs more predictable than other approaches and could make functional programs easier to run on multiple processors.

Understanding its goals

Imperative programming, the kind of programming that most developers have done until now, is akin to an assembly line, where data moves through a series of steps in a specific order to produce a particular result. The process is fixed and rigid, and the person implementing the process must build a new assembly line every time an application requires a new result. Object-oriented programming (OOP) simply modularizes and hides the steps, but the underlying paradigm is the

same. Even with modularization, OOP often doesn't allow rearrangement of the object code in unanticipated ways because of the underlying interdependencies of the code.

REMEMBER Functional programming gets rid of the interdependencies by replacing procedures with pure functions, which requires the use of immutable state. Consequently, the assembly line no longer exists; an application can manipulate data using the same methodologies as those used in pure math. The seeming restriction of immutable state provides the means to allow anyone who understands the math of a situation to also create an application to perform the math.

Using pure functions creates a flexible environment in which code order depends on the underlying math. That math models a real-world environment, and as our understanding of that environment changes and evolves, the math model and functional code can change with it — without the usual problems of brittleness that cause imperative code to fail. Modifying functional code is faster and less error prone because the person implementing the change must understand only the math and doesn't need to know how the underlying code works. In addition, learning how to create functional code can be faster as long as the person understands the math model and its relationship to the real world.

Functional programming also embraces a number of unique coding approaches, such as the capability to pass a function to another function as input. This capability enables you to change application behavior in a predictable manner that isn't possible using other programming paradigms.

Understanding Pure and Impure Languages

Languages that support functional programming fall into two categories: pure and impure. A *pure language* allows only functional programming techniques and fully implements the functional programming paradigm. An *impure language* allows the use of other programming techniques and may only mostly implement the functional programming paradigm. Both pure and impure languages have specific advantages and disadvantages, as described in the sections that follow.

Using the pure approach

Programming languages that use the pure approach to the functional programming paradigm rely on lambda calculus principles, for the most part. In addition, a pure-approach language allows the use of functional programming techniques only, so the result is always a functional program. Haskell is probably the most popular pure language because it provides the purest implementation, according to articles such as the one found on Quora at `https://www.quora.com/What-are-the-most-popular-and-powerful-functional-programming-languages`. Haskell is also a relatively popular language, according to the TIOBE index (`https://www.tiobe.com/tiobe-index/`). Other pure-approach languages include Lisp, Racket, Erlang, and OCaml.

WARNING As with many elements of programming, opinions run strongly regarding whether a particular programming language qualifies for pure status. For example, many people would consider JavaScript to be a pure language, even though it's untyped. Others feel that domain-specific declarative languages such as SQL and Lex/Yacc qualify for pure status even though they aren't general programming languages. Simply having functional programming elements doesn't qualify a language as adhering to the pure approach.

Using the impure approach

Many developers have come to see the benefits of functional programming. However, they also don't want to give up the benefits of their existing language, so they use a language that mixes functional features with one of the other programming paradigms (as described in the "Comparing the Functional Paradigm" section that follows). For example, you can find functional programming features in languages such as C++, C#, and Java. When working with an impure language, you need to exercise care because your code won't work in a purely functional manner, and the features that you might think will work in one way actually work in another. For example, you can't pass a function to another function in some languages.

TIP At least one language, Python, is designed from the outset to support multiple programming paradigms (see https://blog.newrelic.com/2015/04/01/python-programming-styles/ for details). In fact, some online courses make a point of teaching this particular aspect of Python as a special benefit (see https://www.coursehero.com/file/p1hkiub/Python-supports-multiple-programming-paradigms-including-object-oriented/). The use of multiple programming paradigms makes Python quite flexible but also leads to complaints and apologists. This chapter relies on Python to demonstrate the impure approach to functional programming because it's both popular and flexible, plus it's easy to learn.

Comparing the Functional Paradigm

You might think that only a few programming paradigms exist besides the functional programming paradigm explored in this chapter, but the world of development is literally packed with them. That's because no two people truly think completely alike. Each paradigm represents a different approach to the puzzle of conveying a solution to problems by using a particular methodology while making assumptions about

things like developer expertise and execution environment. In fact, you can find entire sites that discuss the issue, such as the one at `https://cs.lmu.edu/~ray/notes/paradigms/`. Oddly enough, some languages (such as Python) mix and match compatible paradigms to create an entirely new way to perform tasks based on what has happened in the past.

REMEMBER The following sections discuss just four of these other paradigms. These paradigms are neither better nor worse than any other paradigm, but they represent common schools of thought. Many languages in the world today use just these four paradigms, so your chances of encountering them are quite high.

Imperative

Imperative programming takes a step-by-step approach to performing a task. The developer provides commands that describe precisely how to perform the task from beginning to end. During the process of executing the commands, the code also modifies application state, which includes the application data. The code runs from beginning to end. An imperative application closely mimics the computer hardware, which executes machine code. *Machine code* is the lowest set of instructions that you can create and is mimicked in early languages, such as assembler.

Procedural

Procedural programming implements imperative programming, but adds functionality such as code blocks and procedures for breaking up the code. The compiler or interpreter still ends up producing machine code that runs step by step, but the use of procedures makes it easier for a developer to follow the code and understand how it works. Many procedural languages provide a disassembly mode in which you can

see the correspondence between the higher-level language and the underlying assembler. Examples of languages that implement the procedural paradigm are C and Pascal.

TECHNICAL STUFF Early languages, such as Basic, used the imperative model because developers creating the languages worked closely with the computer hardware. However, Basic users often faced a problem called *spaghetti code,* which made large applications appear to be one monolithic piece. Unless you were the application's developer, following the application's logic was often hard. Consequently, languages that follow the procedural paradigm are a step up from languages that follow the imperative paradigm alone.

Object-oriented

The procedural paradigm does make reading code easier. However, the relationship between the code and the underlying hardware still makes it hard to relate what the code is doing to the real world. The object-oriented paradigm uses the concept of objects to hide the code, but the more important aim is to make modeling the real world easier. A developer creates code objects that mimic the real-world objects they emulate. These objects include properties, methods, and events to allow the object to behave in a particular manner. Examples of languages that implement the object-oriented paradigm are C++ and Java.

REMEMBER Languages that implement the object-oriented paradigms also implement both the procedural and imperative paradigms. The fact that objects hide the use of these other paradigms doesn't mean that a developer hasn't written code to create the object using these older paradigms. Consequently, the object-oriented paradigm still relies on

code that modifies application state, but could also allow for modifying variable data.

Declarative

Functional programming actually implements the declarative programming paradigm, but the two paradigms are separate. Other paradigms, such as logic programming, implemented by the Prolog language, also support the declarative programming paradigm. The short view of declarative programming is that it does the following:

- Describes what the code should do, rather than how to do it
- Defines functions that are referentially transparent (without side effects)
- Provides a clear correspondence to mathematical logic

Using Python for Functional Programming Needs

Remember that functional programming is a paradigm, which means that it doesn't have an implementation. The basis of functional programming is lambda calculus (`https://brilliant.org/wiki/lambda-calculus/`), which is actually a math abstraction. Consequently, when you want to perform tasks by using the functional programming paradigm, you're really looking for a programming language that implements functional programming in a manner that meets your needs. In fact, you may even be performing functional programming tasks in your current language without realizing it. Every time you create and use a lambda function, you're likely using functional programming techniques (in an impure way, at least).

In addition to using lambda functions, languages like Python that implement the functional programming paradigm have some other features in common. Here is a quick overview of these features:

- **First-class and higher-order functions:** Both first-class and higher-order functions allow you to provide a function as an input, as you would when using a higher-order function in calculus.
- **Pure functions:** A pure function has no side effects. When working with a pure function, you can
  - Remove the function if no other functions rely on its output
  - Obtain the same results every time you call the function with a given set of inputs
  - Reverse the order of calls to different functions without any change to application functionality
  - Process the function calls in parallel without any consequence
  - Evaluate the function calls in any order, assuming that the entire language doesn't allow side effects
- **Recursion:** Functional language implementations rely on recursion to implement looping. In general, recursion works differently in functional languages because no change in application state occurs.
- **Referential transparency:** The value of a variable (a bit of a misnomer because you can't change the value) never changes in a functional language implementation because functional languages lack an assignment operator.

REMEMBER You often find a number of other considerations for performing tasks in functional programming language implementations, but these issues aren't consistent across languages. For example, some languages use strict (eager) evaluation, while other languages use non-strict (lazy) evaluation. Under strict evaluation, the language fully checks the function before evaluating it. Even when a term within the function isn't used, a failing term will cause the function as a whole to fail. However, under non-strict evaluation, the function fails only if the failing term is used to create an output. The Miranda, Clean, and Haskell languages all implement non-strict evaluation.

Various functional language implementations also use different type systems, so the manner in which the underlying computer detects the type of a value changes from language to language. In addition, each language supports its own set of data structures. These kinds of issues

aren't well defined as part of the functional programming paradigm, yet they're important to creating an application, so you must rely on the language you use to define them for you. Assuming a particular implementation in any given language is a bad idea because it isn't well defined as part of the paradigm.

Understanding How Functional Data Works

Data is a representation of something — perhaps a value. However, it can just as easily represent a real-world object. The data itself is always abstract, and existing computer technology represents it as a number. Even a character is a number: The letter *A* is actually represented as the number 65. The letter is a value, and the number is the representation of that value: the data. The following sections discuss data with regard to how it functions within the functional programming paradigm.

Working with immutable data

Being able to change the content of a variable is problematic in many languages. The memory location used by the variable is important. If the data in a particular memory location changes, the value of the variable pointing to that memory location changes as well. The concept of immutable data requires that specific memory locations remain untainted.

REMEMBER Python data isn't immutable in all cases. The "Passing by reference versus by value" section that appears later in the chapter gives you an example of this issue. When working with Python code, you can rely on the `id` function to help you determine when changes have occurred to variables. For example, in the following code, the output of the comparison between `id(x)` and `oldID` will be false:

```
x = 1
```

```
oldID = id(x)
```

```
x = x + 1
```

```
id(x) == oldID
```

WARNING Every scenario has some caveats, and doing this with Python does as well. The `id` of a variable is always guaranteed unique except in certain circumstances:

- One variable goes out of scope and another is created in the same location.
- The application is using multiprocessing and the two variables exist on different processors.
- The interpreter in use doesn't follow the CPython approach to handling variables.

When working with other languages, you need to consider whether the data supported by that language is actually immutable and what set of events occurs when code tries to modify that data. When working with Python, you can detect changes, but not all languages support the functionality required to ensure that immutability is maintained.

Considering the role of state

Application *state* is a condition that occurs when the application performs tasks that modify global data. An application doesn’t have state when using functional programming. The lack of state has the positive effect of ensuring that any call to a function will produce the same results for a given input every time, regardless of when the application calls the function. However, the lack of state has a negative effect as well: The application now has no memory. When you think about

state, think about the capability to remember what occurred in the past, which, in the case of an application, is stored as global data.

Eliminating side effects

The term *declaration* has a number of meanings in computer science, and different people use the term in different ways at different times. For example, in the context of a language such as C, a declaration is a language construct that defines the properties associated with an identifier. You see declarations used for defining all sorts of language constructs, such as types and enumerations. However, that's not how the functional paradigm uses the term *declaration.* When making a functional declaration, you're telling the underlying language to do something. The declaration "Make me a cup of tea!" has only one output: the cup of tea. The declaration doesn't describe how to make the tea; the assumption is that the recipient of the declaration knows how to perform the task.

A *procedure* details what to do, when to do it, and how to do it. Nothing is left to chance and no knowledge is assumed on the part of the recipient. The steps appear in a specific order, and performing a step out of order will cause problems. For example, given the procedure for making a cup of tea, imagine pouring the hot water over the teabag before placing the teabag in the cup. Procedures are often error prone and inflexible, but they do allow for precise control over the execution of a task. Even though making a declaration might seem to be superior to a procedure, using procedures does have advantages that you must consider when designing an application.

The procedure has a *side effect* instead of a value. After making a cup of tea, the procedure indicates that the recipient of the request should take the cup of tea to the requestor. However, the procedure must successfully conclude for this event to occur. The procedure isn't returning the tea; the recipient of the request is performing that task. Consequently, the procedure isn't returning a value.

Side effects also occur in data. When you pass a variable to a function, the expectation in functional programming is that the variable's data will remain untouched — immutable. A side effect occurs when the function modifies the variable data so that upon return from the function call, the variable changes in some manner.

Passing by reference versus by value

The point at which Python shows itself to be an impure language is the use of passing by reference. When you pass a variable by reference, it means that any change to the variable within the function results in a global change to the variable's value. In short, using passing by reference produces a side effect, which isn't allowed when using the functional programming paradigm.

Normally, you can write functions in Python that don't cause the passing by reference problem. For example, the following code doesn't modify `x`, even though you might expect it to:

```
def DoChange(x, y):
    x = x.__add__(y)
    return x
x = 1
print(x)
print(DoChange(x, 2))
print(x)
```

The value of `x` outside the function remains unchanged:

```
1
```

```
3
```

```
1
```

However, you need to exercise care when creating functions using some objects and built-in methods. For example, the following code will modify the output:

```
def DoChange(aList):
    aList.append(4)
    return aList
aList = [1, 2, 3]
print(aList)
print(DoChange(aList))
print(aList)
```

The following output shows that `aList` doesn't remain the same:

```
[1, 2, 3]
[1, 2, 3, 4]
[1, 2, 3, 4]
```

REMEMBER The appended version will become permanent in this case because the built-in function, `append`, performs the modification. To avoid this problem, you must create a new variable within the func-

tion, change its value, and then return the new variable, as shown in the following code:

```
def DoChange(aList):
    newList = aList.copy()
    newList.append(4)
    return newList
aList = [1, 2, 3]
print(aList)
print(DoChange(aList))
print(aList)
```

Here are the new results:

```
[1, 2, 3]
[1, 2, 3, 4]
[1, 2, 3]
```

TIP In the first case, you see the changed list, but the second case keeps the list intact. Whether you encounter a problem with particular Python objects depends on their mutability. An `int` isn't mutable, so you don't need to worry about having problems with functions changing its value. On the other hand, a `list` is mutable, which is the source of the problems with the examples that use a `list` in this section. The article at `https://medium.com/@meghamohan/mutable-and-`

`immutable-side-of-python-c2145cf72747` offers insights into the mutability of various Python objects.

Working with Lists and Strings

After you have used lists, you might be tempted to ask what a list can't do. The list data structure is the most versatile offering for most languages. In most cases, lists are simply a sequence of values that need not be of the same type. You access the elements in a list using an index that begins at 0 for most languages, but could start at 1 for some. The indexing method varies among languages, but accessing specific values using an index is common. Besides storing a sequence of values, you sometimes see lists used in these coding contexts:

- Stack
- Queue
- Deque
- Sets

Generally, lists offer more manipulation methods than other kinds of data structures simply because the rules for using them are so relaxed. Many of these manipulation methods give lists a bit more structure for use in meeting specialized needs. Lists are also easy to search and to perform various kinds of analysis. The point is that lists often offer significant flexibility at the cost of absolute reliability and dependability. (You can easily use lists incorrectly, or create scenarios in which lists can actually cause an application to crash, such as when you add an element of the wrong type.)

TIP Depending on the language you use, lists can provide an impressive array of features and make conversions between types easier. For example, using an iterator in Python lets you perform tasks such as outputting the list as a tuple, processing the content one element at

a time, and unpacking the list into separate variables. The list features you obtain with a particular language depend on the functions the language provides and your own creativity in applying them.

LIST AND ARRAY DIFFERENCE

At first, lists may simply seem to be another kind of array. Many people wonder how lists and arrays differ. After all, from a programming perspective, the two can sound like the same thing. It's true that lists and arrays both store data sequentially, and you can often store any sort of data you want in either structure (although arrays tend to be more restrictive).

The main difference comes in how arrays and lists store the data. An array always stores data in sequential memory locations, which gives an array faster access times in some situations but also slows the creation of arrays. In addition, because an array must appear in sequential memory, updating arrays is often hard, and some languages don't allow you to modify arrays in the same ways as you can lists.

A list stores data using a linked data structure in which a list element consists of the data value and one or two pointers. Lists take more memory because you must now allocate memory for pointers to the next data location (and to the previous location as well in doubly-linked lists, which is the kind used by most languages today). Lists are often faster to create and add data to because of the linking mechanism, but they provide slower read access than arrays.

Creating lists

Python, like most programming languages, makes creating lists easy. You use the following code to create a list:

```
a = [1, 2, 3, 4]
```

The variable `a` now contains a list of four values from `1` to `4`. You can also create a list in Python based on a range. Here is one method for creating a list in Python based on a range:

```
b = list(range(1, 13))
```

This example combines the `list` function with the `range` function to create the list. Notice that the `range` function accepts a starting value, `1`, and a stop value, `13`. The resulting list will contain the values 1 through 12 because the stop value is always one more than the actual output value, as shown by this code:

```
print(b)
```

Here is the result you see:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

Python supports list comprehensions, but again, the code for creating a list in this manner is different. Here's an example of how you could create the list, `c`, using list comprehensions:

```
c = [a * 2 for a in range(1,5)]
```

```
print(c)
```

TIP This example shows the impure nature of Python because you rely on a statement rather than lambda calculus to get the job done. As an alternative, you can define the range function stop value by specifying `len(a)+1`. (The alternative approach makes it easier to create a list based on comprehensions because you don't have to remember the source list length.) Here is the output from this example:

```
[2, 4, 6, 8]
```

Evaluating lists

Python provides a lot of different ways to evaluate lists. To start with, you can obtain a particular element using an index enclosed in square brackets. For example, assuming that you have a list defined as `a = [1, 2, 3, 4, 5, 6]`, typing **a[0]** and pressing Enter will produce an output of 1. To interact with lists in Python, you use modifications of an index, as shown here:

- `a[0]`: Obtains the head of the list, which is 1 in this case
- `a[1:]`: Obtains the tail of the list, which is `[2,3,4,5,6]` in this case
- `a[:-1]`: Obtains all but the last element, which is `[1,2,3,4,5]` in this case
- `a[-1:]`: Obtains just the last element, which is `6` in this case
- `a[:-3]`: Requires the number of elements you want to see as input and then shows that number from the beginning of the list, which is [1,2,3] in this case
- `a[-3:]`: Requires the number of elements you don't want to see as input and then shows the remainder of the list after dropping the required elements, which is [4,5,6] in this case

Python probably provides more ways to slice and dice lists than you'll ever need or want. You can also perform similar levels of basic analysis using Python, as shown here:

- `len(a)`: Returns the number of elements in a list.
- `not a`: Checks for an empty list. This check is different from `a is None`, which checks for an actual null value — `a` not being defined.
- `min(a)`: Returns the smallest list element.
- `max(a)`: Returns the largest list element.
- `sum(a)`: Adds the numbers of the list together.

Interestingly enough, Python has no single method call to obtain the product of a list — that is, all the numbers multiplied together. Python relies heavily on third-party libraries such as NumPy (`https://numpy.org/`) to perform this task. One of the easiest ways to

obtain a product without resorting to a third-party library is shown here:

```
from functools import reduce

print(reduce(lambda x, y: x * y, a))
```

Using `a`, the output is `720` in this case. The `reduce` method found in the `functools` library (see `https://docs.python.org/3/library/functools.html` for details) is incredibly flexible in that you can define almost any operation that works on every element in a list. In this case, the lambda function multiplies the current list element, `y`, by the accumulated value, `x`. If you wanted to encapsulate this technique into a function, you could do so using the following code:

```
prod = lambda z: reduce(lambda x, y: x * y, z)

print(prod(a))
```

As before, you obtain an output of `720`. Python does provide you with a number of statistical calculations in the `statistics` library (see `https://pythonprogramming.net/statistics-python-3-module-mean-standard-deviation/` for details). However, you may find that you want to create your own functions to determine things like the average value of the entries in a list. The following code shows the Python version:

```
avg = lambda x: sum(x) // len(x)

print(avg(a))
```

As before, the output is `3`. Note the use of the `//` operator to perform integer division. If you were to use the standard division operator, you would receive a floating-point value as output.

Performing common list manipulations

List manipulation means changing the list. However, in the functional programming paradigm, you can't change anything. For all intents and purposes, every variable points to a list that is a constant — one that can't change for any reason whatsoever. So when you work with lists in functional code, you need to consider the performance aspects of such a requirement. Every change you make to any list will require the creation of an entirely new list, and you have to point the variable to the new structure. To the developer, the list may appear to have changed, but underneath, it hasn't; in fact, it can't, or the underlying reason to use the functional programming paradigm fails. With this caveat in mind, here are the common list manipulations you want to consider (which are in addition to the evaluations described earlier):

- **Concatenation:** Adding two lists together to create a new list with all the elements of both
- **Repetition:** Creating a specific number of duplicates of a source list
- **Membership:** Determining whether an element exists within a list and potentially extracting it
- **Iteration:** Interacting with each element of a list individually
- **Editing:** Removing specific elements, reversing the list in whole or in part, inserting new elements in a particular location, sorting, or in any other way modifying a part of the list while retaining the remainder

When working with Python, you have access to a whole array of list manipulation functions. Many of them are dot functions that you append to a list. Of course, using the dot functions is fine if you want to modify your original list, but in many situations, modifying the original idea is simply a bad idea, so you need another way to accomplish the task. In this case, you can use the following code to reverse a list and place the result in another list without modifying the original:

```
reverse = lambda x: x[::-1]
```

```
b = reverse(a)
```

```
print(b)
```

Using `a` from the previous sections, you see an output of

```
[6, 5, 4, 3, 2, 1]
```

TIP Python provides an amazing array of list functions — too many to cover in this chapter (but you do see more as the book progresses). One of the best places to find a comprehensive list of Python list functions is at `https://likegeeks.com/python-list-functions/`.

Understanding the Dict and Set alternatives

Python supports both dictionaries and sets. To create a dictionary, you provide name value pairs, as shown here:

```
myDict = {"First": 1, "Second": 2, "Third": 3}
```

The first value, the name, is also a key. The keys are separated from the values by a colon; individual entries are separated by commas. You can access any value in the dictionary using the key, such as `print(myDict["First"])`, which outputs a value of `1`.

Sets in Python are either mutable (the `set` object) or immutable (the `frozenset` object). The immutability of the `frozenset` allows you to use it as a subset within another set or make it hashable for use in a dictionary. (The `set` object doesn't offer these features.) Python has other kinds of sets, too, but for now, the focus is on immutable sets for functional programming uses. Be aware, however, that other set types exist that may work better for your particular application. The following code creates a `frozenset`:

```
myFSet = frozenset([1, 2, 3, 4, 5, 6])
```

You use the `frozenset` to perform math operations or to act as a list of items. For example, you could create a set consisting of the days of the week. You can't locate individual values in a `frozenset` but rather must interact with the object as a whole. However, the object is iterable, so the following code tells you whether `myFSet` contains the value `1`:

```
for entry in myFSet:
   if entry == 1:
      print(True)
```

Considering the use of strings

Strings convey thoughts in human terms. Humans don't typically speak numbers or math; they use strings of words made up of individual letters to convey thoughts and feelings. Unfortunately, computers don’t know what a letter is, much less strings of letters used to create words or groups of words used to create sentences. None of it makes sense to computers. So, as foreign as numbers and math might be to most humans, strings are just as foreign to the computer (if not more so).

Humans see several kinds of objects as strings, but computer languages usually treat them as separate entities. Two of them are important for programming tasks in this book: characters and strings. A *character* is a single element from a character set, such as the letter *A*. Character sets can contain nonletter components, such as the carriage return control character. Extended character sets can provide access to letters used in languages other than English. However, no matter how someone structures a character set, a character is always a single entity within that character set. Depending on how the originator structures the character set, an individual character can consume 7, 8, 16, or even 32 bits.

A *string* is a sequential grouping of zero or more characters from a character set. When a string contains zero elements, it appears as an empty string. Most strings contain at least one character, however. The representation of a character in memory is relatively standard across languages; it consumes just one memory location for the specific size of that character. Strings, however, appear in various forms depending on the language. So computer languages treat strings differently from characters because of how each of them uses memory.

Strings don't just see use as user output in applications. Yes, you do use strings to communicate with the user, but you can also use strings for other purposes such as labeling numeric data within a dataset. Strings are also central to certain data formats, such as XML. In addition, strings appear as a kind of data. For example, HTML relies on the agent string to identify the characteristics of the client system. Consequently, even if your application doesn't ever interact with the user, you're likely to use strings in some capacity.

Python, as an impure language, also comes with a full list of string functions — too many to go into in this chapter. Creating a string is exceptionally easy:

```
myString = "Hello There!"
```

Strings are first-class citizens in Python, and you have access to all the usual manipulation features found in other languages, including special formatting and escape characters. (The tutorial at `https://www.tutorialspoint.com/python/python_strings.htm` doesn't even begin to show you everything, but it's a good start.)

REMEMBER An important issue for Python developers is that strings are immutable. Of course, that leads to all sorts of questions relating to how someone can seemingly change the value of a string in a variable.

However, what really happens is that when you change the contents of a variable, Python actually creates a new string and points the variable to that string rather than the existing string.

TIP One of the more interesting aspects of Python is that you can also treat strings sort of like lists. The "Evaluating lists" section, later in this chapter, talks about how to evaluate lists, and many of the same features work with strings. You have access to all the indexing features to start with, but you can also do things like the following:

- `min(myString)`: Returns the space in the example string
- `max(myString)`: Returns the letter *r* in the example string

Obviously, you can't use `sum(myString)` because there is nothing to sum. In fact, you get a `TypeError` exception if you try. With Python, if you're not quite sure whether something will work on a string, give it a try.

Employing Pattern Matching

Patterns consist of a set of qualities, properties, or tendencies that form a characteristic or consistent arrangement — a repetitive model. Humans are good at seeing strong patterns everywhere and in everything. In fact, we purposely place patterns in everyday things, such as wallpaper or fabric. However, computers are better than humans are at seeing weak or extremely complex patterns because computers have the memory capacity and processing speed to do so. The capability to see a pattern is *pattern matching*. Pattern matching is an essential component in the usefulness of computer systems and has been from the outset, so this chapter is hardly about something radical or new. Even so, understanding how computers find patterns is incredibly important in defining how this seemingly old technology plays

such an important part in new applications such as AI, machine learning, deep learning, and data analysis of all sorts.

The most useful patterns are those that we can share with others. To share a pattern with someone else, you must create a language to define it — an *expression.* This chapter also discusses regular expressions, a particular kind of pattern language, and their use in performing tasks such as data analysis. The creation of a regular expression helps you describe to an application what sort of pattern it should find, and then the computer, with its faster processing power, can locate the precise data you need in a minimum amount of time. This basic information helps you understand more complex pattern matching of the sort that occurs within the realms of AI and advanced data analysis.

Of course, working with patterns using pattern matching through expressions of various sorts works a little differently in the functional programming paradigm. The final sections of this chapter look at how to perform pattern matching using the two languages for this book: Haskell and Python. These examples aren't earth shattering, but they do give you an idea of just how pattern matching works within functional programs so that you can apply pattern matching to other uses.

Looking for patterns in data

When you look at the world around you, you see patterns of all sorts. The same holds true for data that you work with, even if you aren't fully aware of seeing the pattern at all. For example, telephone numbers and social security numbers are examples of data that follows one sort of pattern — that of a positional pattern. A telephone number in the United States consists of an area code of three digits, an exchange of three digits (even though the exchange number is no longer held by a specific exchange), and an actual number within that exchange of four digits. The positions of these three entities is important to the formation of the telephone number, so you often see a tele-

phone number pattern expressed as (999) 999-9999 (or some variant), where the value 9 is representative of a number. The other characters provide separation between the pattern elements to help humans see the pattern.

Other sorts of patterns exist in data, even if you don't think of them as such. For example, the arrangement of letters from *A* to *Z* is a pattern. This statement may not seem like a revelation, but the use of this particular pattern occurs almost constantly in applications when the application presents data in ordered form to make it easier for humans to understand and interact with the data. Organizational patterns are essential to the proper functioning of applications today, yet humans take them for granted, for the most part.

Another sort of pattern is the progression. One of the easiest and most often applied patterns in this category is the exponential progression expressed as N^x, where a number N is raised to the x power. For example, an exponential progression of 2 starting with 0 and ending with 4 would be: 1, 2, 4, 8, and 16. The language used to express a pattern of this sort is the algorithm, and you often use programming language features, such as recursion, to express it in code.

Some patterns are abstractions of real-world experiences. Consider color, for example. To express color in terms that a computer can understand requires the use of three or four three-digit variables, where the first three are always some value of red, blue, and green. The fourth entry can be an alpha value, which expresses opacity, or a gamma value, which expresses a correction used to define a particular color with the display capabilities of a device in mind. These abstract patterns help humans model the real world in the computer environment so that still other forms of pattern matching can occur (along with other tasks, such as image augmentation or color correction).

Transitional patterns help humans make sense of other data. For example, referencing all data to a known base value enables you to com-

pare data from different sources, collected at different times and in different ways, using the same scale. Knowing how various entities collect the required data provides the means for determining which transition to apply to the data so that it can become useful as part of a data analysis.

Data can even have patterns when missing or damaged. The pattern of unusable data could signal a device malfunction, a lack of understanding of how the data collection process should occur, or even human behavioral tendencies. The point is that patterns occur in all sorts of places and in all sorts of ways, which is why having a computer recognize them can be important. Humans may see only part of the picture, but a properly trained computer can potentially see them all.

REMEMBER So many kinds of patterns exist that documenting them all fully would easily take an entire book. Just keep in mind that you can train computers to recognize and react to data patterns automatically in such a manner that the data becomes useful to humans in various endeavors. The automation of data patterns is perhaps one of the most useful applications of computer technology today, yet very few people even know that the act is taking place. What they see instead is an organized list of product recommendations on their favorite site or a map containing instructions on how to get from one point to another — both of which require the recognition of various sorts of patterns and the transition of data to meet human needs.

Understanding regular expressions

Regular expressions are special strings that describe a data pattern. The use of these special strings is so consistent across programming languages that knowing how to use regular expressions in one language makes it significantly easier to use them in all other languages that support regular expressions. As with all reasonably flexible and

feature-complete syntaxes, regular expressions can become quite complex, which is why you'll likely spend more than a little time working out the precise manner by which to represent a particular pattern to use in pattern matching.

REMEMBER You use regular expressions to refer to the technique of performing pattern matching using specially formatted strings in applications. However, the actual code class used to perform the technique appears as Regex, regex, or even RegEx, depending on the language you use. Some languages use a different term entirely, but they're in the minority. Consequently, when referring to the code class rather than the technique, use Regex (or one of its other capitalizations).

TIP The following sections constitute a brief overview of regular expressions. You can find the more detailed Python documentation at `https://docs.python.org/3.6/library/re.html`. This source of additional help can become quite dense and hard to follow, though, so you might also want to review the tutorial at `https://www.regular-expressions.info/` for further insights.

CAPITALIZATION MATTERS!

When working with regular expressions, you must exercise extreme care in capitalizing the pattern correctly. For example, telling the regular expression compiler to look for a lowercase *a* excludes an uppercase *A*. To look for both, you must specify both.

The same holds true when defining control characters, anchors, and other regular expression pattern elements. Some elements may have both lowercase and uppercase equivalents. For example, \w may spec-

ify any word character, while \W specifies any nonword character. The difference in capitalization is important.

Defining special characters using escapes

Character escapes usually define a special character of some sort, very often a control character. You escape a character using the backslash (\), which means that if you want to search for a backslash, you must use two backslashes in a row (\\). The character in question follows the escape. Consequently, \b signals that you want to look for a backspace character. Programming languages standardize these characters in several ways:

- **Control character:** Provides access to control characters such as tab (\t), newline (\n), and carriage return (\r). Note that the \n character (which has a value of \u000D) is different from the \r character (which has a value of \u000A).
- **Numeric character:** Defines a character based on numeric value. The common types include octal (*nnn*), hexadecimal (\x*nn*), and Unicode (\u*nnnn*). In each case, you replace the *n* with the numeric value of the character, such as `\u0041` for a capital letter *A* in Unicode. Note that you must supply the correct number of digits and use 0s to fill out the code.
- **Escaped special character:** Specifies that the regular expression compiler should view a special character, such as (or [, as a literal character rather than as a special character. For example, `\(` would specify an opening parenthesis rather than the start of a subexpression.

Defining wildcard characters

A wildcard character can define a kind of character, but never a specific character. You use wildcard characters to specify any digit or any character at all. The following list tells you about the common wildcard characters. Your language may not support all these characters, or it may define characters in addition to those listed. Here's what the following characters match with:

Character	*Matches With*
.	Any character (with the possible exception of the newline character or other control characters)
\w	Any word character
\W	Any nonword character
\s	Any whitespace character
\S	Any non-whitespace character
\d	Any decimal digit
\D	Any nondecimal digit

Working with anchors

Anchors define how to interact with a regular expression. For example, you may want to work with only the start or end of the target data. Each programming language appears to implement some special conditions with regard to anchors, but they all adhere to the basic syntax (when the language supports the anchor). The following table defines the commonly used anchors:

Anchor	*What It Does*
^	Looks at the start of the string.
$	Looks at the end of the string.

Anchor	*What It Does*
*	Matches zero or more occurrences of the specified character.
+	Matches one or more occurrences of the specified character. The character must appear at least once.
?	Matches zero or one occurrences of the specified character.
{*m*}	Specifies *m* number of the preceding characters required for a match.
{*m*,*n*}	Specifies the range from *m* to *n,* which is the number of the preceding characters required for a match.
expression|*expression*	Performs or searches where the regular expression compiler will locate either one expression or the other expression and count it as a match.

REMEMBER You may find figuring out some of these anchors difficult. The idea of matching means to define a particular condition that meets a demand. For example, consider this pattern: h?t, which would match *hit* and *hot,* but not *hoot* or *heat,* because the ? anchor matches just one character. If you instead wanted to match *hoot* and *heat* as well, then you'd use h*t, because the * anchor can match multiple characters. Using the right anchor is essential to obtaining a desired result.

Delineating subexpressions using grouping constructs

A grouping construct tells the regular expression compiler to treat a series of characters as a group. For example, the grouping construct `[a-z]` tells the regular expression compiler to look for all lowercase characters between *a* and *z*. However, the grouping construct `[az]` (without the dash between *a* and *z*) tells the regular expression compiler to look for just the letters *a* and *z*, but nothing in between, and the grouping construct `[^a-z]` tells the regular expression compiler to look for everything but the lowercase letters *a* through *z*. The following list describes the commonly used grouping constructs. The italicized letters and words in this list are placeholders.

Construct	***What It Means***
[*x*]	Look for a single character from the characters specified by *x*.
[*x*-*y*]	Search for a single character from the range of characters specified by *x* and *y*.
[^*expression*]	Locate any single character not found in the *character expression*.
(*expression*)	Define a regular *expression* group. For example, `ab{3}` would match the letter *a* and then three copies of the letter *b*, that is, `abbb`. However, `(ab){3}` would match three copies of the expression ab: `ababab`.

Using pattern matching in analysis

Pattern matching in computers is as old as the computers themselves. In looking at various sources, you can find different starting points for pattern matching, such as editors. However, the fact is that you can't really do much with a computer system without having some sort of pattern matching occur. For example, the mere act of stopping certain kinds of loops requires that a computer match a pattern between the existing state of a variable and the desired state. Likewise, user input requires that the application match the user's input to a set of acceptable inputs.

Developers recognize that function declarations also form a kind of pattern and that to call the function successfully, the caller must match the pattern. Sending the wrong number or types of variables as part of the function call causes the call to fail. Data structures also form a kind of pattern because the data must appear in a certain order and be of a specific type.

REMEMBER Where you choose to set the beginning for pattern matching depends on how you interpret the act. Certainly, pattern matching isn't the same as counting, as in a `for` loop in an application. However, someone could argue that testing for a condition in a `while` loop matches the definition of pattern matching to some extent. Many people look at editors as the first use of pattern matching because editors were the first kinds of applications to use pattern matching to perform a search, such as to locate a name in a document. Searching is most definitely part of the act of analysis because you must find the data before you can do anything with it.

The act of searching is just one aspect, however, of a broader application of pattern matching in analysis. The act of filtering data also requires pattern matching. A search is a singular approach to pattern matching in that the search succeeds the moment that the application locates a match. Filtering is a batch process that accepts all the

matches in a document and discards anything that doesn't match, enabling you to see all the matches without doing anything else. Filtering can also vary from searching in that searching generally employs static conditions, while filtering can employ some level of dynamic condition, such as locating the members of a set or finding a value within a given range.

Filtering is the basis for many of the analysis features in declarative languages, such as SQL, when you want to locate all the instances of a particular data structure (a record) in a large data store (the database). The level of filtering in SQL is much more dynamic than in mere filtering because you can now apply conditional sets and limited algorithms to the process of locating particular data elements.

TIP Regular expressions, although not the most advanced of modern pattern-matching techniques, offer a good view of how pattern matching works in modern applications. You can check for ranges and conditional situations, and you can even apply a certain level of dynamic control. Even so, the current master of pattern matching is the algorithm, which can be fully dynamic and incredibly responsive to particular conditions.

Working with pattern matching

Pattern matching in Python closely matches the functionality found in many other languages. Python provides robust pattern-matching capabilities using the regular expression (`re`) library (`https://docs.python.org/3.6/library/re.html`). The resource at `https://www.regular-expressions.info/python.html` provides a good overview of the Python capabilities. The following sections detail Python functionality using a number of examples.

Performing simple Python matches

All the functionality you need for employing Python in basic RegEx tasks appears in the `re` library. The following code shows how to use this library:

```
import re

vowels = "[aeiou]"

print(re.search(vowels,
               "This is a test sentence.").group())
```

The `search()` function locates only the first match, so you see the letter `i` as output because it's the first item in `vowels`. You need the `group()` function call to output an actual value because `search()` returns a `match` object, as described at `https://docs.python.org/3.6/library/re.html#match-objects`.

When you look at the Python documentation, you find quite a few functions devoted to working with regular expressions, some of them not entirely clear in their purpose. For example, you have a choice between performing a `search` or a `match`. A `match` works only at the beginning of a string. Consequently, this code:

```
print(re.match(vowels, "This is a test sentence."))
```

returns a value of `None` because none of the vowels appears at the beginning of the sentence. However, this code:

```
print(re.match("a", "abcde").group())
```

returns a value of `a` because the letter *a* appears at the beginning of the test string.

REMEMBER Neither `search` nor `match` will locate all occurrences of the pattern in the target string. To locate all the matches, you use `findall` or `finditer` instead. For example, this code:

```
print(re.findall(vowels, "This is a test sentence."))
```

returns a list like this:

```
['i', 'i', 'a', 'e', 'e', 'e', 'e']
```

Because this is a list, you can manipulate it as you would any other list.

TECHNICAL STUFF Match objects are useful in other ways. For example, you can create a more complete search by using the `start()` and `end()` functions, as shown in the following code:

```
testSentence = "This is a test sentence."

m = re.search(vowels, testSentence)

while m:

   print(testSentence[m.start():m.end()])

   testSentence = testSentence[m.end():]

   m = re.search(vowels, testSentence)
```

This code keeps performing searches on the remainder of the sentence after each search until it no longer finds a match, as shown here:

```
i
```

```
i
```

```
a
```

```
e
```

```
e
```

```
e
```

```
e
```

Using the `finditer()` function would be easier, but this code points out that Python does provide everything needed to create relatively complex pattern-matching code.

Doing more than matching

Python's regular expression library makes it quite easy to perform a wide variety of tasks that don't strictly fall into the category of pattern matching. This chapter discusses only a few of the more interesting capabilities. One of the most commonly used is splitting strings. For example, you might use the following code to split a test string using a number of whitespace characters:

```
testString = "This is\ta test string.\nYippee!"
```

```
whiteSpace = "[\s]"
```

```
print(re.split(whiteSpace, testString))
```

The escaped character, `\s`, stands for all space characters, which includes the set of `[ \t\n\r\f\v]`. The `split()` function can split any content using any of the accepted regular expression characters, so it's an extremely powerful data manipulation function. The output from this example looks like this:

```
['This', 'is', 'a', 'test', 'string.', 'Yippee!']
```

Performing substitutions using the `sub()` function is another forte of Python. Rather than perform common substitutions one at a time, you can perform them all simultaneously, as long as the replacement value is the same in all cases. Consider the following code:

```
testString = "Stan says hello to Margot from Estoria."

pattern = "Stan|hello|Margot|Estoria"

replace = "Unknown"

re.sub(pattern, replace, testString)
```

The output of this example is

```
Unknown says Unknown to Unknown from Unknown.
```

You can create a pattern of any complexity and use a single replacement value to represent each match. This is handy when performing certain kinds of data manipulation for tasks such as dataset cleanup prior to analysis.

Working with Recursion

Some people confuse recursion with a kind of looping. The two are completely different sorts of programming and wouldn't even look the same if you could view them at a low level. In *recursion,* a function calls itself repetitively and keeps track of each call through stack entries, rather than an application state, until a condition used to determine the need to make the function call meets some requirement. At this point, the list of stack entries unwinds with the function passing the results of its part of the calculation to the caller until the stack is empty and the initiating function has the required output of the call.

Although this sounds mind-numbingly complex, in practice, recursion is an extremely elegant method of solving certain computing problems and may be the only solution in some situations. The following sections introduce you to the basics of recursion using Python, so don't worry if this initial definition leaves you in doubt as to what recursion means.

Performing tasks more than once

One of the main advantages of using a computer is its capability to perform tasks repetitively — often far faster and with greater accuracy than a human can. Even a language that relies on the functional programming paradigms requires some method of performing tasks more than once; otherwise, creating the language wouldn't make sense. Because the conditions under which functional languages repeat tasks differ from those of other languages using other paradigms, thinking about the whole concept of repetition again is worthwhile, even if you've worked with these other languages. The following sections give you a brief overview.

Defining the need for repetition

The act of repeating an action seems simple enough to understand. However, repetition in applications occurs more often than you might think. Here are just a few uses of repetition to consider:

- Performing a task a set number of times
- Performing a task a variable number of times until a condition is met
- Performing a task a variable number of times until an event occurs
- Polling for input
- Creating a message pump
- Breaking a large task into smaller pieces and then executing the pieces
- Obtaining data in chunks from a source other than the application
- Automating data processing using various data structures as input

In fact, you could easily create an incredibly long list of repeated code elements in most applications. The point of repetition is to keep from

writing the same code more than once. Any application that contains repeated code becomes a maintenance nightmare. Each routine must appear only once to make its maintenance easy, which means using repetition to allow execution more than one time.

Using recursion instead of looping

The functional programming paradigm doesn't allow the use of loops for two simple reasons. First, a loop requires the maintenance of state, and the functional programming paradigm doesn't allow state. Second, loops generally require mutable variables so that the variable can receive the latest data updates as the loop continues to perform its task. As mentioned previously, you can't use mutable variables in functional programming. These two reasons would seem to sum up the entirety of why to avoid loops, but there is yet another.

One of the reasons that functional programming is so amazing is that you can use it on multiple processors without concern for the usual issues found with other programming paradigms. Because each function call is guaranteed to produce precisely the same result, every time, given the same inputs, you can execute a function on any processor without regard to the processor use for the previous call. This feature also affects recursion because recursion lacks state.

When a function calls itself, it doesn't matter where the next function call occurs; it can occur on any processor in any location. The lack of state and mutable variables makes recursion the perfect tool for using as many processors as a system has to speed applications as much as possible.

Understanding recursion

Recursion, in its essence, is a method of performing tasks repetitively, wherein the original function calls itself. Various methods are available for accomplishing this task, as described in the following sections.

The important aspect to keep in mind, though, is the repetition. Whether you use a list, dictionary, set, or collection as the mechanism to input data is less important than the concept of a function's calling itself until an event occurs or it fulfills a specific requirement. Basic recursion is the kind that you normally see demonstrated for most languages. In this case, the `doRep` function creates a list containing a specific number, `n`, of a value, `x`, as shown here for Python:

```
def doRep(x, n):
   y = []
   if n == 0:
      return []
   else:
      y = doRep(x, n - 1)
      y.append(x)
      return y

print(doRep(4, 5))
```

To understand this code, you must think about the process in reverse. The code begins with a call to `doRep()` with `x = 4` and `n = 5`. Before it does anything else, the code actually calls `doRep()` repeatedly until `n == 0`. So, when `n == 0`, the first actual step in the recursion process is to create an empty list, which is what the code does.

At this point, the call returns and the first actual step concludes, even though you have called `doRep()` six times before it gets to this point.

The next actual step, when `n == 1`, is to make `y` equal to the first actual step, an empty list, and then append `x` to the empty list by calling `y.append(x)`. At this point, the second actual step concludes by returning `[4]` to the previous step, which has been waiting in limbo all this time. The recursion continues to unwind until `n == 5`, at which point it performs the final append and returns `[4, 4, 4, 4, 4]` to the caller.

TIP Sometimes it's incredibly hard to wrap your head around what happens with recursion, so putting a `print` statement in the right place can help. Here's a modified version of the Python code with the `print` statement inserted. Note that the `print` statement goes after the recursive call so that you can see the result of making it.

```
def doRep(x, n):
   y = []
   if n == 0:
      return []
   else:
      y = doRep(x, n - 1)
      print(y)
      y.append(x)
      return y
```

```
print(doRep(4, 5))
```

This version of the code returns the following:

```
[]
```

```
[4]
```

```
[4, 4]
```

```
[4, 4, 4]
```

```
[4, 4, 4, 4]
```

```
[4, 4, 4, 4, 4]
```

Using recursion on lists

Lists represent multiple inputs to the same call during the same execution. A list can contain any data type in any order. You use a list when a function requires more than one value to calculate an output. For example, consider the following Python list:

```
myList = [1, 2, 3, 4, 5]
```

If you wanted to use standard recursion to sum the values in the list and provide an output, you could use the following code:

```
def lSum(list):
   if not list:
      return 0
   else:
      return list[0] + lSum(list[1:])
```

```
print(lSum(myList))
```

You see an output of `15` in this case. The function relies on slicing to remove one value at a time from the list and add it to the sum. The *base case* (principle, simplest, or foundation) is that all the values are gone and now `list` contains the empty set, ([]), which means that it has a value of 0.

Using lambda functions in Python recursion isn't always easy, but this particular example lends itself to using a lambda function quite easily. The advantage is that you can create the entire function in a single line, as shown in the following code (with two lines, but using a line-continuation character):

```
lSum2 = lambda list: 0 if not list \
   else list[0] + lSum2(list[1:])

print(lSum2(myList))
```

As before, the result is `15`. The code works precisely the same as the longer example, relying on slicing to get the job done.

Considering advanced recursive tasks

You have full access to the various data structures in Python when using functional programming techniques. For example, a *dictionary* takes the exclusivity of sets one step further by creating key/value pairs, in which the key is unique but the value need not be. Using keys makes searches faster because the keys are usually short and you need only to look at the keys to find a particular value. Python places the

key first, followed by the value. Here's a dictionary definition in Python:

```
myDic = {"a": 1, "b": 2, "c": 3, "d": 4}
```

You can access the individual values by using the key. Python uses a form of index to access individual values, such as `myDic["b"]`, which also accesses the value `2`. You can use recursion with dictionaries in the same manner as you do lists. However, recursion really begins to shine when it comes to complex data structures. Consider this Python nested dictionary:

```
myDic = {"A":{"A": 1, "B":{"B": 2, "C":{"C": 3}}}, "D": 4}
```

In this case, you have a dictionary nested within other dictionaries down to four levels, creating a complex dataset. In addition, the nested dictionary contains the same `"A"` key value as the first-level dictionary (which is allowed), the same `"B"` key value as the second level, and the "C" key on the third level. You might need to look for the repetitious keys, and recursion is a great way to do that, as shown here:

```
def findKey(obj, key):
   for k, v in obj.items():
      if isinstance(v, dict):
         findKey(v, key)
      else:
         if key in obj:
            print(obj[key])

```

```
print(findKey(myDic, "B"))
```

This code looks at all the entries by using a `for` loop. Notice that the loop unpacks the entries into key, `k`, and value, `v`. When the value is another dictionary, the code recursively calls `findKey` with the value and the key as input. Otherwise, if the instance isn't a dictionary, the code checks to verify that the key appears in the input object and prints just the value of that object. In this case, an object can be a single entry or a sub-dictionary. Here is the output from this example:

```
{'B': 2, 'C': {'C': 3}}
```

```
2
```

```
None
```

Passing functions instead of variables

Being able to pass a function to another function provides much needed flexibility. The passed function can modify the receiving function's response without modifying that receiving function's execution. The two functions work in tandem to create output that's an amalgamation of both.

TIP Normally, when you use this function-within-a-function technique, one function determines the process used to produce an output, while the second function determines how the output is achieved. This isn't always the case, but when creating a function that receives another function as input, you need to have a particular goal in mind that actually requires that function as input. Given the complexity of debugging this sort of code, you need to achieve a specific level of flexibility by using a function rather than some other input.

Also tempting is to pass a function to another function to mask how a process works, but this approach can become a trap. Try to execute the function externally when possible and input the result instead. Otherwise, you might find yourself trying to discover the precise location of a problem rather than processing data.

For the example in this section, you can't use a lambda function to perform the required tasks with Python, so the following code relies on standard functions instead:

```
def doAdd(x, y):
   return x + y

def doSub(x, y):
   return x - y

def compareWithHundred(function, x, y):
   z = function(x, y)
   out = lambda x: "GT" if 100 > x \
      else "EQ" if 100 == x else "LT"
   return out(z)

print(compareWithHundred(doAdd, 99, 2))
print(compareWithHundred(doSub, 99, 2))
```

```
print(compareWithHundred(doAdd, 99, 1))
```

This example outputs one of three strings: `GT` when 100 > x; EQ when 100 == x; and LT when 100 < x. To use `compareWithHundred()`, you pass either `doAdd()` or `doSub()`, which simply adds to or subtracts from the second argument, `x`, the third argument, `y`. To make the comparison, `compareWithHundred()` uses a lambda function, which appears on two lines in this case. The output from this example shows how the comparisons work:

```
LT
```

```
GT
```

```
EQ
```

Performing Functional Data Manipulation

The act of manipulating data means to modify it in some controlled way. You want some of the data, but not all of it. Data analysis often hinges on manipulating data in specific ways. The rest of the book does show various sorts of data manipulation, but the following sections provide you with a quick start.

Slicing and dicing

In some respects, slicing and dicing is considerably easier in Python than in some other languages. For one thing, you use indexes to perform the task. Also, Python offers more built-in functionality. Consequently, the one-dimensional list example looks like this:

```
myList = [1, 2, 3, 4, 5]
```

```

```

```
print(myList[:2])

print(myList[2:])

print(myList[2:3])
```

The use of indexes enables you to write the code succinctly and without using special functions. The output is as you would expect:

```
[1, 2]

[3, 4, 5]

[3]
```

Slicing a two-dimensional list is every bit as easy as working with a one-dimensional list. Here's the code and output for the two-dimensional part of the example:

```
myList2 = [[1,2],[3,4],[5,6],[7,8],[9,10]]

print(myList2[:2])

print(myList2[2:])

print(myList2[2:3])
```

Here’s the output:

```
[[1, 2], [3, 4]]

[[5, 6], [7, 8], [9, 10]]

[[5, 6]]
```

Dicing does require using a special function, but the function is concise in this case and doesn't require multiple steps:

```
def dice(lst, rb, re, cb, ce):
    lstr = lst[rb:re]
    lstc = []
    for i in lstr:
        lstc.append(i[cb:ce])
    return lstc
```

To call `dice()`, you need to provide a two-dimensional list (`lst`), the beginning row (`rb`), ending row (`re`), beginning column (`cb`), and the ending column (`ce`). In this case, you can't really use a lambda function — or not easily, at least. The code slices the incoming list first and then dices it, but everything occurs within a single function. Notice that Python requires the use of looping, but this function uses a standard `for` loop instead of relying on recursion. The disadvantage of this approach is that the loop relies on state, which means that you can't really use it in a fully functional setting. Here's the test code for the dicing part of the example:

```
myList3 = [[1,2,3],[4,5,6],[7,8,9],[10,11,12],[13,14,15]]

print(dice(myList3, 1, 4, 1, 2))
```

Here's the output:

```
[[5], [8], [11]]
```

Mapping your data

You can find a number of extremely confusing references to the term *map* in computer science. For example, a map is associated with database management (see `https://en.wikipedia.org/wiki/Data_mapping`), in which data elements are mapped between two distinct data models. However, for this chapter, *mapping* refers to a process of applying a high-order function to each member of a list. Because the function is applied to every member of the list, the relationships among list members is unchanged. Many reasons exist to perform mapping, such as ensuring that the range of the data falls within certain limits. The following code provides an example of mapping:

```
square = lambda x: x**2

double = lambda x: x + x

items = [0, 1, 2, 3, 4]


print(list(map(square, items)))

print(list(map(double, items)))
```

The output shows that you get a square or double of each value in the list, as shown here:

```
[0, 1, 4, 9, 16]

[0, 2, 4, 6, 8]
```

Note that you must convert the `map` object to a `list` object before printing it. Given that Python is an impure language, creating code

that processes a list of inputs against two or more functions is relatively easy, as shown in this code:

```
funcs = [square, double]

for i in items:
   value = list(map(lambda items: items(i), funcs))
   print(value)
```

Here is the output from this example:

```
[0, 0]
[1, 2]
[4, 4]
[9, 6]
[16, 8]
```

Filtering data

Python doesn't provide the niceties of other languages when it comes to filtering. For example, you don’t have access to special keywords, such as odd or even. In fact, all the filtering in Python requires the use of lambda functions. To perform filtering, you use code like this:

```
items = [0, 1, 2, 3, 4, 5]

print(list(filter(lambda x: x % 2 == 1, items)))
```

```
print(list(filter(lambda x: x > 3, items)))
```

```
print(list(filter(lambda x: x % 3 == 0, items)))
```

which results in the following output:

```
[1, 3, 5]
```

```
[4, 5]
```

```
[0, 3]
```

Notice that you must convert the `filter` output using a function such as `list`. You don't have to use `list`; you could use any data structure, including `set` and `tuple`. The lambda function you create must evaluate to `True` or `False`.

Organizing data

Placing data in a particular order makes it easier to perform tasks like seeing patterns. A computer can often use unsorted data, but humans need sorted data to make sense of it. The examples in this section use the following list:

```
original = [(1, "Hello"), (4, "Yellow"), (5, "Goodbye"),
            (2, "Yes"), (3, "No")]
```

To understand these examples, you need to know how to use the `sort()` method versus the `sorted()` function. When you use the `sort()` method, Python changes the original list, which may not be what you want. In addition, `sort()` works only with lists, while `sorted()` works with any iterable. The `sorted()` function produces output that doesn't change the original list. Consequently, if you want to maintain your original list form, you use the following call:

```
sorted(original)
```

The output is sorted by the first member of the tuple: `[(1, 'Hello'), (2, 'Yes'), (3, 'No'), (4, 'Yellow'), (5, 'Goodbye')]`, but the original list remains intact. Reversing a list requires the use of the `reverse` keyword, as shown here:

```
sorted(original, reverse=True)
```

Python can make use of lambda functions to perform special sorts. For example, to sort by the second element of the tuple, you use the following code:

```
sorted(original, key=lambda x: x[1])
```

TIP The `key` keyword is extremely flexible. You can use it in several ways. For example, `key=str.lower` would perform a case-insensitive sort. Some of the common lambda functions appear in the `operator` module. For example, you could also sort by the second element of the tuple using this code:

```
from operator import itemgetter
```

```
sorted(original, key=itemgetter(1))
```

You can also create complex sorts. For example, you can sort by the length of the second tuple element by using this code:

```
sorted(original, key=lambda x: len(x[1]))
```

REMEMBER Notice that you must use a lambda function when performing a custom sort. For example, trying this code will result in an error:

```
sorted(original, key=len(itemgetter(1)))
```

Even though `itemgetter` is obtaining the key from the second element of the tuple, it doesn't possess a length. To use the second tuple's length, you must work with the tuple directly.

Table of contents collapsed